

Desafio de performance: Custo de ordenação

Assignment: Trabalho Prático

Vitor C. Santos¹, Pedro G. A. Oliveira², João P. Miranda³, João V. Nunes⁴, Juan Rikelme⁵

¹Faculdade da Saúde e Ecologia Humana (FASEH)

Rua São Paulo, 958, Parque Jardim Alterosa – CEP 33200-000 – Vespasiano – MG – Brasil

{972410828, 972412169, 972310486, 972311029, 972311105}@ulife.com.br

Resumo. Objetivo: Implementar e comparar o desempenho de um algoritmo de ordenação de complexidade $O(n^2)$ (simples) contra um de complexidade $O(n \log n)$ (eficiente). **Metodologia ou Etapas:** A organização deste resumo estruturado segue uma adaptação da ABNT 6028:2021, que versa sobre normas para apresentação de resumos.

Palavras-Chave Algoritmos de Ordenação, Insertion Sort, Merge Sort, Complexidade Computacional, Análise de Desempenho.

1. Introdução

Este trabalho apresenta o desenvolvimento, a implementação manual (sem bibliotecas prontas) e o confronto empírico de dois paradigmas de ordenação em arrays na linguagem Java: o Insertion Sort (complexidade quadrática $O(n^2)$) e o Merge Sort (complexidade eficiente $O(n \log n)$). Através da submissão dos algoritmos a cenários controlados de estresse (Melhor Caso, Caso Médio e Pior Caso) com amostras de 1.000 e 1.000.000 de elementos, a pesquisa quantifica precisamente o impacto prático da análise assintótica sobre o hardware. Os resultados consolidam o entendimento de que para grandes volumes de dados (Big Data), a escolha do algoritmo não afeta apenas o tempo, mas dita a própria viabilidade operacional do software.

2. Funcionamento

2.1. Algoritmo Simples: Insertion Sort

O Insertion Sort é um algoritmo baseado em comparação, o algoritmo divide o array em duas partes: uma sublista já ordenada (que começa apenas com o primeiro elemento) e uma sublista de elementos não ordenados. A partir do segundo elemento (índice 1), o algoritmo consome uma "chave" (chave = array[i]), a chave é comparada com os elementos da sublista ordenada (da direita para a esquerda). Enquanto os elementos ordenados forem maiores que a chave, eles são empurrados uma posição para a direita para abrir espaço. Assim que encontra um elemento menor ou igual à chave (ou o início do array), o algoritmo posiciona a chave na vaga liberada. O processo se repete para cada elemento do array até que todo o conjunto esteja na sublista ordenada. No pior caso (dados em ordem decrescente) e no caso médio (dados aleatórios), sua complexidade é $O(n^2)$. Por isso, ele é chamado de algoritmo simples ou menos eficiente para grandes volumes de dados.

2.2. Algoritmo Eficiente: Merge Sort

O algoritmo calcula o ponto médio do array e o divide em duas metades. Esse processo de divisão ocorre de forma recursiva até que o array seja fragmentado em subarranjos de tamanho 1. (Um array com apenas 1 elemento é considerado logicamente ordenado). Utilizando o (merge), o algoritmo pega dois subarranjos ordenados e os combinam em um único array maior também ordenado, para isso ele cria arrays temporários para a esquerda e para a direita, compara os menores elementos disponíveis em cada lado e insere o menor deles de volta no array original. Caso sobre elementos em um dos lados (quando um lado "acaba" antes do outro), o restante é copiado diretamente para o final do array. Por isso o Merge Sort garante um tempo de execução de $O(n \log n)$ em todos os casos (pior, médio e melhor caso). Isso o torna extremamente eficiente para arrays maiores.

3. Resultados

Tabela 1. Métricas

Arquivo	Insertion Sort	Merge Sort
dados_pior_1K	1ms	0ms
dados_pior_1M	246.850ms	50ms
dados_medio_1K	1ms	0ms
dados_medio_1M	124.568ms	88ms
dados_melhor_1K	0ms	0ms
dados_melhor_1M	3ms	36ms

4. Análise Crítica

O algoritmo de complexidade $O(n^2)$ demorou muito mais no arquivo grande porque seu tempo de execução cresce muito rapidamente conforme a quantidade de elementos aumenta. No caso do Insertion Sort, em situações médias e de pior caso, o algoritmo precisa comparar e mover muitos elementos várias vezes, fazendo com que o número de operações aumente de forma quadrática. $O(n^2)$

Isso significa que, ao aumentar a entrada, o tempo não cresce proporcionalmente, mas muito mais rapidamente. Por exemplo, ao sair de 1.000 elementos para 1.000.000, a quantidade de comparações pode aumentar drasticamente, tornando a execução muito mais lenta. Já o algoritmo eficiente (Merge Sort) apresenta complexidade: $O(n \log n)$

Nesse caso, o tempo de execução também aumenta quando a entrada cresce, porém de forma muito mais controlada. O Merge Sort divide o problema em partes menores, ordena essas partes e depois junta os resultados, mantendo um desempenho eficiente mesmo com grandes quantidades de dados. Assim, ao aumentar o tamanho do arquivo, o algoritmo eficiente também demora mais, mas o crescimento do tempo é bem menor quando comparado ao algoritmo $O(n^2)$, tornando-o mais adequado para grandes volumes de dados.

Uma curiosidade é que no arquivo de um milhão de dados já ordenado o tempo no algoritmo de complexidade simples foi melhor, pois o merge sort faz a divisão e junção dos arquivos mesmo que eles já estejam ordenados.

5. Conclusão

A realização deste trabalho permitiu consolidar de forma prática e empírica os conceitos teóricos de complexidade de algoritmos, evidenciando como a escolha da estrutura lógica de um código impacta diretamente o uso dos recursos de hardware.

Em suma, o projeto atinge o seu objetivo acadêmico ao demonstrar que o desenvolvimento de software eficiente exige que o desenvolvedor saiba analisar o cenário operacional (tamanho da entrada e restrições de hardware) antes de decidir qual estrutura lógica implementar.

Referências

GeeksforGeeks (2026a). Insertion sort algorithm. <https://www.geeksforgeeks.org/insertion-sort/>. Acesso em: 30 maio 2026.

GeeksforGeeks (2026b). Merge sort algorithm. <https://www.geeksforgeeks.org/merge-sort/>. Acesso em: 30 maio 2026.